카카오 식당 데이터베이스 구축 및 추천 로직 1단계부터 6단계 구현·검증 보고서

SQLite + SQLAlchemy + FastAPI + Kakao Local API + Playwright

작성일 2026년 11월 9일
프로젝트 C:₩TeamProject_1

보고서 개요

이 보고서는 기존 데모 식당 목록을 실제 카카오 장소 검색 결과와 상세페이지 정보로 교체하기 위해 진행한 1단계부터 6단계까지의 작업을 설명한다. 개발 경험이 적은 독자도 이해할 수 있도록 먼저 개념을 설명하고, 그 다음 실제 구현 방식과 검증 명령을 제시한다.

> **한 문장 요약**
> 카카오 API로 주변 식당의 기본정보를 찾고, 상세페이지에서 메뉴·가격·영업시간을 수집한 뒤, 현재 한국 시간과 사용자의 예산을 기준으로 추천 가능 여부를 계산하는 시스템을 구축했다.

구분	내용
목표	실제 식당 데이터 기반의 메뉴·예산·현재 영업 여부 추천
백엔드	FastAPI, Pydantic, SQLAlchemy
현재 DB	SQLite - 추후 Supabase PostgreSQL 전환 예정
외부 데이터	카카오 장소 검색 API, 카카오 장소 상세페이지
동적 페이지	Playwright Chromium으로 JavaScript 렌더링
검증 결과	Python 53개 통과, 프론트엔드 29개 통과, 실제 통합 흐름 확인

전체 시스템 구조

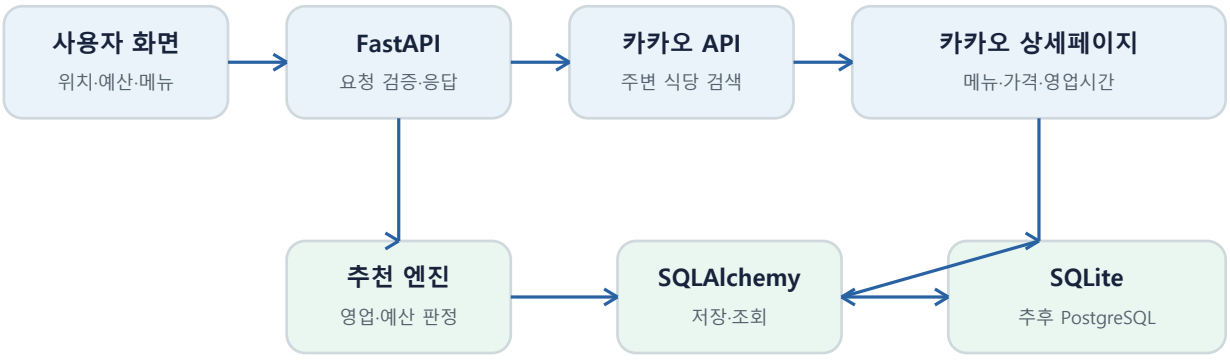


그림 1. 사용자의 추천 요청이 카카오 검색, 상세페이지 수집, DB 저장, 추천 판정으로 이어지는 구조

목차

보고서 개요 2

전체 시스템 구조 . . . 2

목차 3

1. 프로젝트 배경과 핵심 개념 5

초보자를 위한 용어 . . . 5

왜 API와 상세페이지를 나누는가 . . . 5

2. 1단계 - 가능성 검토와 상세페이지 조사 6

확인한 데이터 범위 . . . 6

동적 페이지 확인 결과 . . . 6

1단계 결론 . . . 6

1단계 검증 방법 . . . 6

3. 2단계 - DB 테이블과 관계 설계 7

테이블별 책임 . . . 7

PK와 FK가 같은 이유 . . . 7

영업시간을 여러 행으로 저장하는 이유 . . . 7

2단계 검증 명령 . . . 7

4. 3단계 - SQLite와 SQLAlchemy 연결 9

구현 구성 . . . 9

SQLite 안전 설정 . . . 9

3단계 검증 . . . 9

5. 4단계 - 카카오 식당 기본정보 저장 10

API 필드 매핑 . . . 10

저장 안정성 . . . 10

4단계 검증 . . . 10

6. 5단계 - 상세페이지 메뉴·가격·영업시간 수집 11

수집 순서 . . . 11

정규화 규칙 . . . 11

수집 상태 . . . 11

5단계 검증 . . . 12

7. 6단계 - 실제 추천 조건 적용 12

추천 판정 순서 . . . 12

한국 시간과 익일 영업 . . . 12
브레이크 타임 . . . 13
상세수집 제한과 재사용 . . . 13
6단계 검증 . . . 13
8. 단계별 검증 가이드 13
사전 준비 . . . 13
전체 자동 테스트 . . . 13
서버와 화면 검증 . . . 14
응답 필드 읽는 법 . . . 14
9. 실제 통합 검증 결과 15
결과 해석 . . . 15
DB 상태 확인 명령 . . . 15
10. 코드 구조와 주석 정책 16
테스트 파일 . . . 16
11. 현재 제한사항과 운영 전 보완점 17
권장 다음 단계 . . . 17
12. 결론 18
부록 A. 전체 검증 체크리스트 . . . 18

1. 프로젝트 배경과 핵심 개념

기존 데모는 서버에 미리 작성한 몇 개의 가상 메뉴를 선택하고, 카카오 검색 결과를 단순 후보로 보여주는 구조였다. 실제 식당이 그 메뉴를 판매하는지, 가격이 예산에 맞는지, 현재 영업 중인지는 확인하지 않았다. 이번 작업은 이 세 가지 빈칸을 DB와 상세페이지 수집 로직으로 채우는 것이 핵심이다.

초보자를 위한 용어

용어	쉬운 설명	이 프로젝트의 예
API	다른 서비스에 정해진 형식으로 데이터를 요청하는 창구	카카오에서 주변 음식점을 검색
DB	여러 번 사용할 데이터를 표 형태로 보관하는 공간	식당, 메뉴, 영업시간 저장
PK	한 행을 유일하게 구분하는 값	카카오 식당 ID
FK	다른 테이블의 행을 가리키는 값	menu.restaurant_id
ORM	SQL 대신 Python 객체로 DB를 다루는 방식	SQLAlchemy 모델
Upsert	없으면 추가하고 이미 있으면 갱신하는 처리	같은 카카오 ID 중복 방지
동적 페이지	JavaScript가 실행된 뒤 실제 내용이 나타나는 페이지	카카오 메뉴·영업시간
정규화	표시 문자열을 비교·저장하기 좋은 형식으로 바꾸는 일	9,000원 -> 9000

왜 API와 상세페이지를 나누는가

카카오 장소 검색 API는 ID, 업체명, 분류, 좌표, 주소, 상세 URL을 안정적인 JSON으로 제공한다. 반면 메뉴명, 가격, 요일별 영업시간은 검색 API에 포함되지 않고 상세페이지에만 나타날 수 있다. 따라서 기본정보는 API로 빠르게 저장하고, 상세정보는 별도의 브라우저 수집 과정에서 보완한다.

> **중요한 설계 원칙**
> 검색 API 응답과 상세페이지 수집 결과를 한 번에 처리하지 않는다. 외부 페이지가 느리거나 구조가 바뀌어도 식당 기본정보와 기존에 성공적으로 수집한 상세정보를 보호하기 위해서다.

2. 1단계 - 가능성 검토와 상세페이지 조사

1단계는 코드를 작성하기 전에 데이터가 실제로 어디에서 제공되는지 확인한 조사 단계다. 이 단계에서 잘못된 가정을 제거해야 이후 DB 설계가 흔들리지 않는다.

확인한 데이터 범위

필요 데이터	카카오 검색 API	상세페이지	처리 방식
카카오 ID	제공	URL에도 포함	restaurant PK
업체명·업태	제공	표시됨	API 값을 우선 저장
위도·경도·주소	제공	표시됨	API 값을 저장
메뉴명	미제공	있을 수도 있음	상세 수집 후 menu 저장
가격	미제공	메뉴에 연결될 수 있음	정수 또는 NULL
요일별 영업시간	미제공	있을 수도 있음	영업 구간으로 분리
현재 open/closed	직접 저장 불필요	현재 상태 표시 가능	한국 시간과 DB 시간 비교

동적 페이지 확인 결과

- 일반 HTTP 요청으로는 JavaScript 실행 전의 껍데기 HTML만 얻을 수 있었다.
- 브라우저 렌더링 후 메뉴명과 가격이 나타났고, 영업정보 영역에는 요일별 시간, 브레이크 타임, 라스트오더가 표시됐다.
- 메뉴 더보기를 누르면 메뉴 탭으로 전환되어 홈 탭의 영업시간 DOM이 사라지므로 영업시간을 먼저 읽어야 했다.
- 모든 식당이 메뉴·가격·영업시간을 제공하는 것은 아니므로 '정보 없음'과 '수집 실패'를 구분해야 했다.

1단계 결론

> **구현 가능**
> 기본정보는 카카오 API로 저장할 수 있고, 메뉴·가격·영업시간은 Playwright로 렌더링된 상세페이지에서 수집할 수 있다. 단, 상세페이지 구조 변경 가능성 때문에 수집 상태와 실패 보존 로직이 반드시 필요하다.

1단계 검증 방법

- 카카오 검색 API의 실제 응답에서 id, place_name, category_name, x, y, address_name, place_url을 확인한다.
- 같은 place_url을 일반 HTTP와 브라우저 렌더링 상태에서 비교한다.
- 메뉴가 있는 식당, 메뉴가 없는 식당, 영업시간이 있는 식당, 휴무·브레이크 타임이 있는 식당을 각각 표본으로 확인한다.

3. 2단계 - DB 테이블과 관계 설계

한 식당에는 여러 메뉴와 여러 영업 구간이 존재한다. 이 데이터를 restaurant 한 행에 모두 넣으면 열이 반복되고 검색이 어려워진다. 따라서 역할별로 테이블을 분리했다.

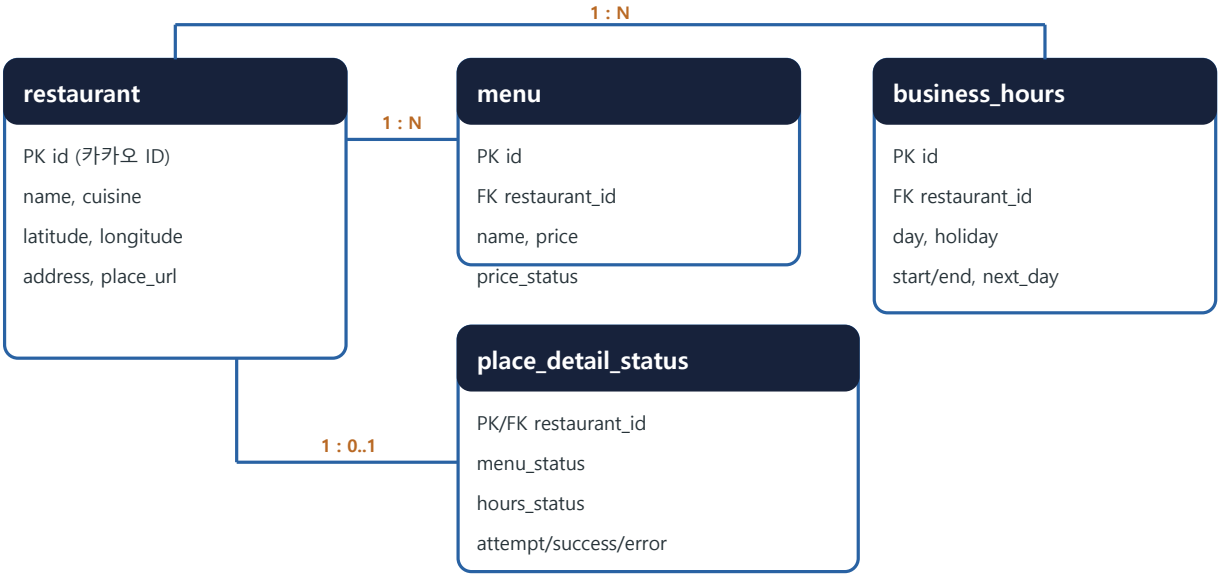


그림 2. 현재 SQLAlchemy 모델 기준 ERD

테이블별 책임

테이블	핵심 열	역할
restaurant	id, name, cuisine, latitude, longitude, address, place_url	카카오 API 기본정보
menu	restaurant_id, name, normalized_name, price, price_status	식당별 메뉴와 가격
business_hours	day, holiday, start/end, ends_next_day, sequence	요일별 실제 영업 구간
place_detail_status	restaurant_id, menu_status, hours_status, attempt/success/error	수집 결과와 재시도 판단

PK와 FK가 같은 이유

place_detail_status.restaurant_id는 PK이면서 FK다. FK이므로 실제 restaurant가 있어야 하고, PK이므로 식당 하나당 상태 행이 최대 하나만 존재한다. 이는 restaurant 1 : place_detail_status 0..1 관계를 가장 단순하게 표현한다.

영업시간을 여러 행으로 저장하는 이유

예를 들어 11:00~21:30 영업 중 15:00~17:00이 브레이크 타임이면 실제 추천 가능한 구간은 11:00~15:00과 17:00~21:30이다. 두 구간을 sequence 0과 1로 저장하면 open/closed 계산에서 브레이크 타임을 별도로 뺄 필요가 없다.

2단계 검증 명령

```
.\.venv\Scripts\python.exe -m pytest tests\test_database.py -v
```

- 4개 테이블이 생성되는지 확인한다.
- FK가 활성화되고 식당 삭제 시 메뉴·영업시간·상태가 함께 삭제되는지 확인한다.

4. 3단계 - SQLite와 SQLAlchemy 연결

SQLite는 별도의 DB 서버를 설치하지 않아도 파일 하나로 동작하므로 프로토타입에 적합하다. SQLAlchemy를 사용하면 나중에 DATABASE_URL을 PostgreSQL 주소로 바꾸고 모델·저장 코드를 대부분 재사용할 수 있다.

구현 구성

구성요소	설명
Settings.database_url	기본값은 data/lunchrun.db, 환경변수 DATABASE_URL로 교체 가능
Database.engine	애플리케이션이 공유하는 SQLAlchemy 연결 엔진
session_factory	요청 또는 작업마다 짧게 생성하는 DB 세션
FastAPI lifespan	서버 시작 시 스키마 생성, 종료 시 엔진 정리
get_db	요청별 세션을 주입하고 응답 후 자동 종료

SQLite 안전 설정

- foreign_keys=ON: SQLite에서 실제 FK 검사를 활성화한다.
- journal_mode=WAL: 읽기와 쓰기의 충돌을 줄인다.
- busy_timeout=5000: 짧은 잠금 충돌은 최대 5초 기다린다.
- check_same_thread=False: FastAPI 스레드풀에서 SQLite 연결을 사용할 수 있게 한다.

3단계 검증

```
.\.venv\Scripts\python.exe -m pytest tests\test_database.py -v
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

서버 시작 후 data/lunchrun.db가 생성되고, 테스트에서 foreign_keys=1 및 journal_mode=wal이 확인되면 정상이다.

> **PostgreSQL 전환 시 주의**
> 테이블 자동 생성은 프로토타입용이다. Supabase PostgreSQL로 전환할 때는 Alembic 마이그레이션을 추가해 스키마 변경 이력을 관리하는 것이 안전하다.

5. 4단계 - 카카오 식당 기본정보 저장

카카오 주변 음식점 검색 결과를 restaurant 테이블에 저장한다. 카카오 ID를 PK로 사용하므로 같은 식당이 반복 검색돼도 행이 중복되지 않는다.

API 필드 매핑

카카오 응답	DB 열	변환
id	restaurant.id	문자열 PK
place_name	name	앞뒤 공백 제거
category_name	category_name / cuisine	원문 저장 + 음식점 아래 첫 분류 추출
y / x	latitude / longitude	문자열을 실수로 변환
road_address_name 또는 address_name	address	도로명 우선
place_url	place_url	상세수집에 사용할 URL

저장 안정성

- category_group_code가 FD6인 음식점만 저장한다.
- 기존 ID가 없으면 insert, 있으면 최신 기본정보로 update한다.
- 각 식당을 SAVEPOINT로 분리해 한 건의 잘못된 데이터가 전체 저장을 취소하지 않게 한다.
- 신규 식당에는 pending 상세수집 상태를 함께 만든다.
- 응답에 inserted, updated, skipped, failed 건수를 포함해 확인할 수 있게 한다.

4단계 검증

```
.\.venv\Scripts\python.exe -m pytest tests\test_restaurant_repository.py tests\test_recommendations.py -v
```

수동 검증에서는 /api/recommendations 응답의 storage 값을 확인하고, 같은 검색을 반복했을 때 inserted 대신 updated가 증가하며 restaurant.id 중복이 생기지 않는지 확인한다.

```
.\.venv\Scripts\python.exe -c "import sqlite3; c=sqlite3.connect(r'data\lunchrun.db'); print(c.execute('SELECT id, COUNT(*) FROM restaurant GROUP BY id HAVING COUNT(*) > 1').fetchall())"
```

결과가 []이면 카카오 ID 중복이 없다.

6. 5단계 - 상세페이지 메뉴·가격·영업시간 수집

상세페이지는 JavaScript 실행 뒤 내용이 나타나므로 Playwright Chromium을 사용한다. 수집, 문자열 정규화, DB 저장을 서로 다른 모듈로 분리해 페이지 구조 변경과 DB 오류의 영향을 줄였다.

수집 순서

순서	처리	이유
1	카카오 ID와 place_url 검증	임의 사이트 접속 방지
2	홈 탭에서 영업시간 먼저 읽기	메뉴 더보기 후 홈 DOM이 사라짐
3	메뉴 더보기 실행 후 전체 메뉴 읽기	홈에는 일부만 표시될 수 있음
4	가격·시간 정규화	DB 검색 가능한 형식으로 변환
5	성공 데이터 교체 또는 실패 상태 기록	중복 방지와 기존 데이터 보호

정규화 규칙

페이지 표시	저장 결과
9,000원	price=9000, price_status=known
시가 / 변동	price=NULL, price_status=variable
가격 없음	price=NULL, price_status=missing
15:00~17:00 브레이크타임	해당 구간을 영업시간에서 제외
18:00~02:00	ends_next_day=True
휴무	holiday=True, 시작·종료 NULL

수집 상태

상태	의미	기존 데이터
pending	아직 시도하지 않음	없음
success	정상 수집	새 결과로 갱신
missing	페이지를 읽었지만 정보 없음	해당 항목을 비움
fetch_failed	페이지 또는 브라우저 접근 실패	보존
parse_failed	페이지 구조 분석 실패	보존

데이터 보호 규칙
수집 실패만으로 기존 메뉴나 영업시간을 삭제하지 않는다. 외부 페이지의 일시 장애가 정상 데이터를 지우는 상황을 방지한다.

5단계 검증

```
.\.venv\Scripts\python.exe -m playwright install chromium
.\.venv\Scripts\python.exe -m pytest tests\test_place_details.py -v
```

실제 표본 검증에서는 카카오 ID 923784754에서 메뉴 40개와 브레이크 타임이 분리된 영업 구간 14개를 저장했고, menu_status와 hours_status가 모두 success임을 확인했다.

7. 6단계 - 실제 추천 조건 적용

6단계는 카카오 검색 결과를 그대로 보여주는 방식을 중단하고, DB에서 메뉴·가격·현재 영업 여부를 확인한 뒤 세 가지 결과로 분류한다.

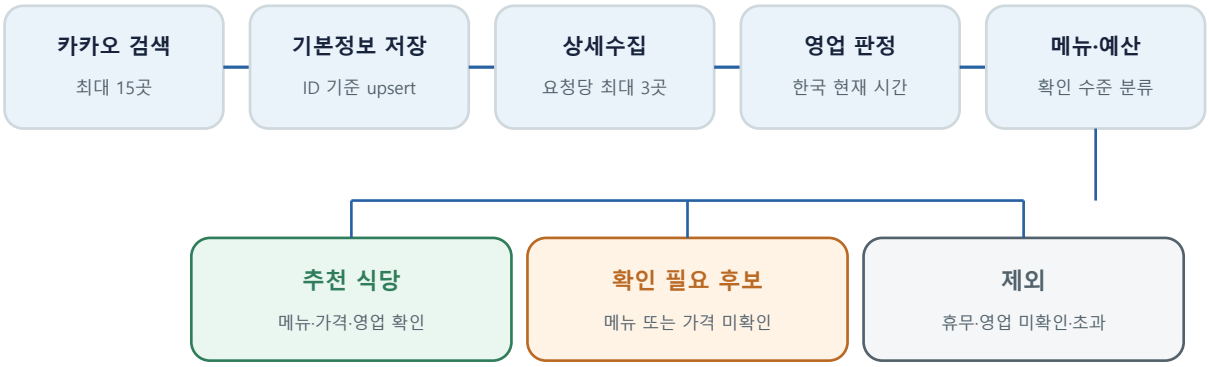


그림 3. 검색 결과가 추천, 확인 필요 후보, 제외로 분류되는 과정

추천 판정 순서

판정	처리
영업시간 status가 success가 아님	현재 상태를 알 수 없으므로 제외
오늘 holiday=True	추천에서 제외
현재 시간이 영업 구간 밖	closed로 제외
영업 중 + 메뉴 미확인	확인 필요 후보, 메뉴·가격 확인 불가 표시
영업 중 + 메뉴 확인 + 가격 NULL	확인 필요 후보, 가격 확인 불가 표시
영업 중 + 가격이 예산 초과	제외
영업 중 + 메뉴 확인 + 가격 예산 이내	추천 식당

한국 시간과 익일 영업

open/closed는 DB에 저장하지 않는다. 요청 시 ZoneInfo('Asia/Seoul')의 현재 요일·시각과 business_hours를 비교한다. 화요일 01:00에는 화요일 구간뿐 아니라 월요일에 시작해 ends_next_day=True인 구간도 확인한다. 오늘이 명시적 휴무라면 전날의 심야 구간보다 휴무를 우선한다.

브레이크 타임

5단계에서 이미 브레이크 타임을 실제 영업 구간에서 제거했기 때문에, 6단계에서는 현재 시각이 저장된 어느 구간에 포함되는지만 확인하면 된다.

상세수집 제한과 재사용

- 신규 검색 결과 중 한 요청에서 최대 3곳만 상세수집한다.
- 정상 수집 결과는 7일 동안 다시 수집하지 않는다.
- 수집 실패는 1시간 후 재시도할 수 있다.
- 같은 검색을 반복하면 이미 수집한 식당을 재사용하고 다음 pending 식당을 수집한다.
- 채식·맵기·총소요시간은 아직 추천 판정에 적용하지 않는다.

6단계 검증

```
.\.venv\Scripts\python.exe -m pytest tests\test_recommendation_engine.py tests\test_recommendations.py
tests\test_place_details.py -v
```

자동 테스트는 예산 초과, 가격 미확인, 메뉴 미확인, 영업시간 미확인, 휴무, 브레이크 타임, 익일 영업, holiday=NULL 경고, 요청당 3곳 제한을 고정 시각으로 검증한다.

8. 단계별 검증 가이드

아래 명령은 프로젝트 루트 C:₩TeamProject_1의 PowerShell에서 실행한다.

사전 준비

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m playwright install chromium
```

단계	핵심 테스트	정상 판단
1	실제 API·상세페이지 표본 확인	기본정보와 동적 상세정보의 출처 구분
2	tests/test_database.py	테이블·FK·cascade 통과
3	DB 파일 생성·PRAGMA 확인	foreign_keys=1, WAL
4	tests/test_restaurant_repository.py	insert/update/skip/fail 집계
5	tests/test_place_details.py	가격·휴무·브레이크·익일·실패보존
6	tests/test_recommendation_engine.py	예산·open/closed·후보 분리

전체 자동 테스트

```
.\.venv\Scripts\python.exe -m pytest -q
node --test tests\groq-status.test.cjs tests\location.test.cjs tests\location-flow.test.cjs
tests\restaurant-ui.test.cjs tests\map.test.cjs
```

현재 확인된 결과
Python: 53 passed, 1 skipped, 37 subtests passed / 프론트엔드: 29 passed. 1개의 Starlette TestClient 사용 중단 예정 경고는 기능 실패가 아니다.

서버와 화면 검증

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload
```

- 브라우저에서 http://127.0.0.1:8000을 연다.
- 위치, 반경, 예산, 음식 종류를 설정하고 추천을 실행한다.
- Network 탭의 /api/recommendations 응답에서 restaurants, candidate_restaurants, excluded, storage, collection을 확인한다.
- 지도에는 restaurants만 표시되고 candidate_restaurants는 별도 확인 필요 영역에 표시되는지 확인한다.

응답 필드 읽는 법

필드	의미
restaurants	메뉴·예산·현재 영업이 확인된 추천 목록
candidate_restaurants	영업 중이지만 메뉴 또는 가격 확인이 필요한 후보
excluded.closed	현재 영업시간 밖 또는 휴무
excluded.hours_unavailable	영업시간 미확인으로 제외
excluded.over_budget	확인 가격이 예산 초과
collection.attempted	이번 요청의 상세수집 수, 최대 3
storage.inserted/updated	restaurant 기본정보 추가·갱신 수

9. 실제 통합 검증 결과

개별 단위 테스트뿐 아니라 실제 카카오 API와 상세페이지를 사용해 전체 흐름을 확인했다.

검증	결과
5단계 상세페이지 표본	메뉴 40개, 영업 구간 14개, 상태 success/success
6단계 실제 주변 검색	식당 15곳 신규 저장
요청당 상세수집 제한	3곳 시도, 3곳 성공
실행 시점 추천 결과	추천 0곳, 확인 필요 후보 0곳
제외 사유	closed 3곳, hours_unavailable 12곳

결과 해석

추천이 0곳이라는 결과는 오류가 아니다. 상세수집한 3곳은 실행 당시 영업시간 밖이었고, 나머지 12곳은 한 요청당 3곳 제한 때문에 아직 영업시간을 확인하지 못해 안전하게 제외됐다. 같은 검색을 반복하면 다음 pending 식당이 최대 3곳씩 수집된다.

DB 상태 확인 명령

```
.\.venv\Scripts\python.exe -c "import sqlite3; c=sqlite3.connect(r'data\lunchrun.db'); print(*c.execute('SELECT menu_status, hours_status, COUNT(*) FROM place_detail_status GROUP BY menu_status, hours_status'), sep='\n')"

.\.venv\Scripts\python.exe -c "import sqlite3; c=sqlite3.connect(r'data\lunchrun.db'); print('  :',c.execute('SELECT COUNT(*) FROM menu').fetchone()[0]); print('    :',c.execute('SELECT COUNT(*) FROM business_hours').fetchone()[0])"
```

10. 코드 구조와 주석 정책

2단계부터 6단계에서 추가한 주요 코드에는 한국어 모듈 설명, 함수 목적, 안전 처리 이유, 분기별 판정 기준을 주석으로 기록했다. 단순히 '무엇을 한다'가 아니라 '왜 그렇게 처리하는가'를 이해할 수 있도록 작성했다.

경로	책임
app/database/models.py	ERD에 해당하는 ORM 모델과 제약조건
app/database/session.py	엔진·세션·SQLite PRAGMA
app/repositories/restaurants.py	카카오 식당 기본정보 upsert
app/collectors/kakao_place.py	Playwright 상세페이지 수집
app/place_details.py	메뉴·가격·영업시간 정규화
app/repositories/place_details.py	상세정보와 상태 저장
app/detail_collection.py	수집 작업·재시도·배치 제한
app/recommendation_engine.py	현재 영업·메뉴·예산 판정
app/services.py	검색·저장·수집·추천 흐름 연결
frontend/app.js	추천과 확인 필요 후보 분리 표시

테스트 파일

파일	검증 범위
tests/test_database.py	스키마·FK·cascade
tests/test_restaurant_repository.py	기본정보 추가·갱신·부분 실패
tests/test_place_details.py	URL·파싱·실패 보존·수집 제한
tests/test_recommendation_engine.py	예산·영업시간·휴무·익일 추천
tests/test_recommendations.py	FastAPI 추천·채팅 통합
tests/restaurant-ui.test.cjs	추천·후보 화면 분리와 안전 출력

11. 현재 제한사항과 운영 전 보완점

항목	현재 상태	권장 보완
상세페이지 구조	CSS 선택자 변화 가능	수집 실패율 모니터링과 선택자 테스트
수집 실행	추천 요청 안에서 최대 3곳 처리	Redis 작업 큐와 백그라운드 워커
DB	SQLite 파일	Supabase PostgreSQL + Alembic
데이터 최신성	정상 7일, 실패 1시간	업종별 갱신주기와 관리자 재수집
메뉴 일치	정규화된 부분 문자열	동의어·대표메뉴 사전 또는 검색 인덱스
조건	메뉴·예산·현재 영업	채식·맵기·알레르기 데이터 확보 후 적용
시간	영업 구간과 휴무	임시휴무·공휴일·라스트오더 정책 보강
수집 정책	공개 페이지 최소 수집	서비스 약관·robots·접근량 정책 검토
인증	데모 로그인	Supabase Auth와 RLS

운영 전 가장 중요한 변경
상세페이지 수집을 사용자 요청에서 분리해 Redis 큐와 별도 워커에서 처리하고, API는 이미 수집된 DB 결과를 빠르게 조회하도록 바꾸는 것이 우선이다.

권장 다음 단계

- Alembic을 도입해 SQLite와 PostgreSQL 스키마 변경 이력을 관리한다.
- Redis 큐에 pending 식당을 넣고 Playwright 워커가 속도 제한을 지키며 수집한다.
- 수집 성공률, 메뉴 수, 영업시간 수, 오류 코드별 건수를 모니터링한다.
- 추천 화면에서 place_url을 안전하게 열어 사용자가 상세정보를 직접 확인할 수 있게 한다.
- 공휴일·임시휴무와 라스트오더를 추천 정책에 반영한다.

12. 결론

1단계에서는 카카오 데이터의 실제 제공 범위를 확인했고, 2단계에서는 식당·메뉴·영업시간·수집상태를 분리한 ERD를 설계했다. 3단계에서 SQLite와 SQLAlchemy를 FastAPI에 연결했으며, 4단계에서 카카오 검색 결과를 ID 기준으로 안전하게 저장했다.

5단계에서는 Playwright로 동적 상세페이지를 읽어 메뉴·가격·영업시간을 정규화하고, 실패해도 기존 데이터를 보존하는 수집 체계를 만들었다. 6단계에서는 한국 현재 시간, 휴무, 브레이크 타임, 익일 영업, 메뉴, 예산을 사용해 추천·확인 필요 후보·제외를 구분했다.

> **최종 상태**
> 데모의 가상 식당 후보 표시에서 한 단계 발전해, 실제 카카오 식당 기본정보와 상세페이지 수집 데이터에 근거한 추천 파이프라인이 동작한다. 자동 테스트와 실제 통합 검증을 모두 통과했으며, 다음 핵심 과제는 수집 작업의 백그라운드화와 PostgreSQL 전환이다.

부록 A. 전체 검증 체크리스트

- requirements.txt 설치와 Playwright Chromium 설치
- 전체 Python 테스트 53개 통과
- 프론트엔드 테스트 29개 통과
- DB FK와 WAL 활성화
- 카카오 검색 결과의 insert/update 집계 확인
- 상세페이지 메뉴·영업시간 success/missing/failure 확인
- 추천·후보·제외 응답과 지도 표시 확인
- DB나 응답에 API 키·외부 오류 원문이 노출되지 않는지 확인

- 보고서 끝 -